A truthful chart needs more than a passing sampler.

> **SHIP.** The generated-data monitor, accessible repository and deployed build are ready for review within the assignment scope. This exercise explicitly uses generated data; real training control is outside its scope.

Repository: https://github.com/akenoowww/soup-training-observatory
App: https://akenoowww.github.io/soup-training-observatory/

First improvement after SHIP: impose explicit processing/output budgets for extremely fragmented series; if truthful rendering exceeds the budget, ask for a narrower range instead of silently dropping breaks.

1 / What is kept, and what is discarded

Twenty deterministic runs include NaN/Infinity, duplicates, unordered steps, missing steps, zeros, divergence and a 1M-point run. Source steps must be nonnegative safe integers; unsupported rows are rejected and counted. Numeric sorting uses the last received duplicate, including invalid latest values.

A worker retains four Float64 columns (32 MB at 1M). Exact viewport bounds precede first/last/min/max selection in each finite segment and step bucket. No averages or smoothing are used. Invalid values and absent steps break the line. Sign boundaries are retained before reduction, so log masking also breaks at every nonpositive interval. A unique spike survives selection in every containing viewport by construction; isolated finite points get visible markers. Constant zero lines are padded inside the plot rather than clipped at its edge.

Hover and CSV identify sampled observations. Statistics use all unique rows in view; pending/failed ranges show unavailable statistics. Dense timing and lesser peaks within one bucket are discarded. Subpixel gaps can be visually tiny, so missingness counts remain visible. Pathologically many gaps or sign changes exceed the ordinary output budget and remain a declared workload limit.

2 / Executed evidence, including failures

80 unit tests; 46 scenarios × 3 engines = 138 browser executions passed, without skipped/flaky results. Chromium, Firefox and WebKit ran against the production bundle on macOS. Independent oracles exercise 128 malformed streams, 1,152 dataset/range/metric combinations and 105 outlier/boundary cases.

Browser checks inspect actual Canvas spikes, NaN breaks, singleton/zero lines and drawn axis labels/coordinates. They control real Worker reply order through React/query/client, test stale/malformed responses, loading/error scope, drag/reset, keyboard focus, storage, UTC+14/Russian locale, 320/390/768px, DPR2 and touch emulation. Fifteen axe states have zero confirmed violations; mobile-dialog background estimates remain incomplete, with 31 nodes independently checked at contrast 5.27–13.25:1.

A long run found a serious defect after short tests passed: custom cache eviction removed a new query during observer attachment, retaining responses and growing memory to about 495 MB. The mechanism was removed in favor of TanStack's lifecycle-aware 10-second GC. A real-observer regression failed at range 12 before the fix and passed 40 transitions afterward; the UI regression completes 20 viewports with exactly 20 Worker requests per engine. Original failed tests, allocation samples and rejected-build measurements remain in `evidence/round2/`.

3 / Performance in numbers

Current final bundle `app-pYEMV88W.js`; browser 157.0.8103.50, darwin/arm64. Application and worker SHA256 match performance, sustained-use, delivery and all hosted engines. Per fixture mode: 5 first renders, 5 zooms, 15 hovers; full application: 5/15/15, respectively. Medians below. First render includes preparation; zoom/hover span the actual in-page event to matching charts and two paint opportunities, excluding automation transport.

Controlled loss chart	Raw 1M	Main sampling	Worker sampling
First render	603.6 ms	434.0 ms	235.0 ms
Zoom	12.8 ms	28.1 ms	33.6 ms
Hover	15.8 ms	16.1 ms	16.2 ms
Renderer private footprint, one overview	79.0 MB	54.7 MB	58.5 MB

Full app, historical original → current: first render 515.5 → 507.4 ms; zoom 63.9 → 62.1 ms; hover 32.0 → 32.4 ms.
Previous pre-log-fix baseline → current: first render 495.9 → 507.4 ms; zoom 62.8 → 62.1 ms; hover 32.8 → 32.4 ms. Historical browser versions: 156.0.8078.81. Browser version and run order confound build differences; these samples do not establish causes or a faster complete interface.

CPU ×4 applies to the page; worker scheduling is separately calibrated in the same browser. The main-sampled control separates reduction from worker placement. This does not model an equally throttled whole device. MemoryInfra maps actual frame/worker PIDs outside timed runs and excludes spare/browser/GPU processes. Full 3-chart app: 87.9 MB private renderer footprint. Local evidence, not a device SLA or deployment-latency benchmark. Raw samples/traces: evidence/final-build-measurements/performance/; calibration: evidence/final-build-measurements/cpu-scope-calibration.json.

4 / Silent failures and sustained use

- **Staleness:** immutable source revision and recorded time stay separate from view preparation. Old revisions and impossible chronology fail closed; reload never claims new training observations.
- **Races:** run/revision/range/width identify queries, and a response retains its producing viewport. Late replies cannot populate a different run; prior same-run plots stay covered while updating.
- **Numbers:** UTC and explicit numeric formatting are tested under other browser locales/zones. No smoothing hides divergence; log zero exclusions never become invented positive values.

The same final bundle completed 136 active cycles (546 counted actions) across all 20 runs, 180s activity plus 35s idle at normal CPU scheduling, with 0 browser errors. Retained page+worker V8/backing stores: 38.9 MB warm → 45.0 MB maximum checkpoint → 41.4 MB after idle (+2.5 MB net). Explicit GC includes worker source storage; this measures retained allocations, not unforced peak memory or absence of slower leaks.

5 / Surprise, concern, and AI use

The surprising failure was a correctly retained spike that Canvas did not paint when surrounded by gaps. Later, a cache “optimization” passed short tests yet caused runaway retention. The remaining concern is dense visual compression and workloads with extreme missingness. Physical-device ergonomics, screen-reader usability and exhaustive data/zoom coverage are not established by these tests.

AI tools: Codex and Frontend Workbench assisted architecture, implementation, tests and independent reviews. Accepted suggestions included uPlot, workers, extrema sampling and explicit source identity. Corrected or rejected: fabricated trends, mixed-scope summaries, invisible singleton/zero rendering, custom cache eviction and broad CPU/memory assumptions. Original failures remain in the repository.

My manual review (candidate): I manually checked run search, sorting and switching; the spike at step 500,000 (loss 9.4), hover and synchronized three-chart zoom/reset; missing data; and zero-value behavior on linear/log scales. I also checked reload/back/forward persistence, Tab/search/Escape and the 390 px layout.

Codex review and automation: Codex inspected sampling/Canvas source and rendered PDF pages, reconciled raw counts and memory/soak JSON, and checked four timing plus four memory traces. Automated suites supplied the unit, browser and Canvas regression results.